AI / ML FOUNDATIONS • STAGE 1 • COMPUTATIONAL LAYER

NumPy
Master Notes

PART 4

Fancy Indexing + Sorting Arrays

with k-Nearest Neighbours and Big-O explained step by step, from zero

Based on: Python Data Science Handbook, Chapter 2 (Blocks 7 and 8)

Pictures first | Theory | Examples | Step-by-step code | Revision zone

WHAT IS INSIDE

1. Fancy indexing: pick by a list of positions
2. Sorting arrays: sort, argsort, partition
3. k-Nearest Neighbours from zero (pictures + code)
4. Big-O: how work grows with data
5. Revision zone: cheat sheet and trap radar

Slow-paced notes: see the picture, understand the idea, then read the code. No practice questions in this part.

Prepared for Abhijeet • October 2026

How to use these notes

These notes cover the two blocks you studied from the Python Data Science Handbook: **Block 7: Fancy Indexing** and **Block 8: Sorting Arrays**. Every topic follows the same order you asked for: first the **big picture and a real-life example**, then a **picture**, and only then the **code, explained one step at a time** with the exact output you will see in Colab.

Because you said you cannot memorise every function, each topic carries a priority tag so you know where to spend your energy.

Tag	Meaning	What to do
MUST KNOW	You will type this almost every day in AI/ML work.	Practise until your fingers know it.
GOOD TO KNOW	Used regularly, but not every day.	Know what it does and roughly how it looks.
LOOK UP	Rare, or easy to search when needed.	Remember that it exists. Do not memorise.

The coloured boxes you will meet:

BIG PICTURE	The idea in plain words, before any code. Read this first.
ANALOGY	A real-life picture so the idea sticks.
MEMORY TRICK	A short trick to remember something forever.
INTERVIEW POINT	Say or write this in an answer for full marks.
IMPORTANT	Pay attention: easy to get wrong.
COMMON TRAP	A mistake almost every beginner makes. NumPy never warns you about most of these.
TRY IT	Type this into Colab and watch it happen yourself.
WHY IT MATTERS	Where this idea shows up in real AI/ML work.
VERSION NOTE	Newer library versions differ from the book in a small way.
STUDY RULE	Do not read these notes like a story book only. Open Colab, type the examples, change one value at a time and watch what changes. All outputs in these notes were produced by running the code (NumPy 2.4.4), so what you see on the page is what you should see on your screen. The random numbers use a fixed seed so that your Colab matches the page.

Topic map

#	Topic	Why it matters in AI/ML	Priority
1	Fancy indexing: pick by a list of positions	Embedding lookup, batches, picking rows	MUST KNOW
2	1D fancy indexing: the result follows the index shape	Predict any output shape	MUST KNOW
3	2D fancy indexing: pairs versus grid	probs[range(n), labels], sub-matrices	MUST KNOW
4	Combined indexing (fancy + slice + mask)	Feature selection	MUST KNOW
5	Integer index versus Boolean mask	The most common indexing mix-up	MUST KNOW
6	Random selection, train/test split	Every ML project	MUST KNOW
7	Modifying values, repeated-index trap, np.add.at	Counting words and classes	GOOD TO KNOW
8	Binning data with searchsorted	Histograms, grades, buckets	GOOD TO KNOW
9	Block 8 versus DSA sorting	Knowing what NOT to study here	NOTE
10	np.sort, .sort(), np.argsort	Ranking, top-k predictions	MUST KNOW
11	Sorting along an axis	Row-wise / column-wise ordering	MUST KNOW
12	np.partition, np.argpartition	Top-k without a full sort	GOOD TO KNOW
13	k-Nearest Neighbours from zero (Part 3)	KNN, embeddings, RAG search	MUST KNOW
14	Big-O: how work grows with data (Part 4)	Why vectorise, why attention is costly	GOOD TO KNOW

What we left out on purpose: selection sort and bogosort code (pure data-structures-and-algorithms practice, not a NumPy skill), the 1,000,000-point speed benchmark of `np.histogram`, and the Matplotlib plotting lines (we learn Matplotlib later). The Revision Zone lists them again so nothing is hidden from you.

Contents

How to use these notes . . . 2
Topic map . . . 3

1. Fancy Indexing . . . 6

1.1 The big idea: three ways to pick . . . 6
1.2 Fancy indexing on a 1D array . . . 7
1.3 Fancy indexing in 2D: pairs . . . 8
1.4 Fancy indexing in 2D: the grid . . . 9
1.5 Combined indexing . . . 11
1.6 Integer index versus Boolean mask (the trap) . . . 12
1.7 Selecting random points and splitting data . . . 13
1.8 Modifying values with fancy indexing . . . 14
1.9 Binning data with searchsorted . . . 17

2. Sorting Arrays . . . 19

2.1 What Block 8 is, and what it is not . . . 19
2.2 np.sort and x.sort() . . . 19
2.3 np.argsort: the order, not the values . . . 20
2.4 Sorting along an axis . . . 21
2.5 Partial sorts: partition and argpartition . . . 23

3. k-Nearest Neighbours, from zero . . . 25

3.1 The big idea, with a real-life picture . . . 25
3.2 Step 1: the distance between two points . . . 26
3.3 Step 2: the distance from ONE point to ALL points . . . 26
3.4 Steps 3 and 4: sort, take k, vote . . . 27
3.5 Step 5: the distances between EVERY pair of points . . . 27
3.6 Step 6: the one-line version, decoded . . . 28
3.7 Step 7: reading the neighbours with argsort . . . 30
3.8 Step 8: skip the full sort with argpartition, and the final function . . . 32
3.9 Why this matters, and its limits . . . 33

4. Big-O: How Work Grows . . . 35

4.1 The big idea . . . 35
4.2 The growth rates you will meet . . . 36
4.3 The honest limit of Big-O . . . 36

5. Revision Zone 38

5.1 Function cheat sheet 39

5.2 The trap radar 40

5.3 Topic checklist 40

5.4 Left out on purpose, so nothing is hidden 41

PART 1

1 Fancy Indexing

Block 7: picking many elements at once with a list of positions

IN THIS PART YOU WILL LEARN

- ✓ What fancy indexing is, and how it differs from a scalar index and a slice
- ✓ Why the **shape of the result follows the shape of the index**
- ✓ 2D fancy indexing: **pairs** (`X[row, col]`) versus a **grid** (`X[row[:, None], col]`)
- ✓ Mixing fancy indexing with slices and Boolean masks (and the integer-versus-Boolean trap)
- ✓ Random selection, train/test splits, modifying values, `np.add.at`, and binning with `searchsorted`

1.1 The big idea: three ways to pick

You already know two ways to pick from an array: **one position** (`x[3]`) and **one neat block** (`x[2:6]`). Fancy indexing is the third way: you hand NumPy a **list of positions** and it returns all of them at once, in the order you wrote, repeats allowed.

BIG PICTURE

A slice can only grab things that sit **next to each other** (or at regular steps). Fancy indexing has no such limit: it can grab elements from anywhere, in any order, even the same element twice. The list of positions is called the **index array**.

ANALOGY

Booking cinema seats. A scalar index is "give me seat 3". A slice is "give me seats 2 to 5, they are together". Fancy indexing is "give me seats 0, 3 and 7, wherever they are", and you get them back in exactly the order you asked.



Three ways to pick from the same array. Highlighted cells are what comes back.

PYTHON • type this in Colab

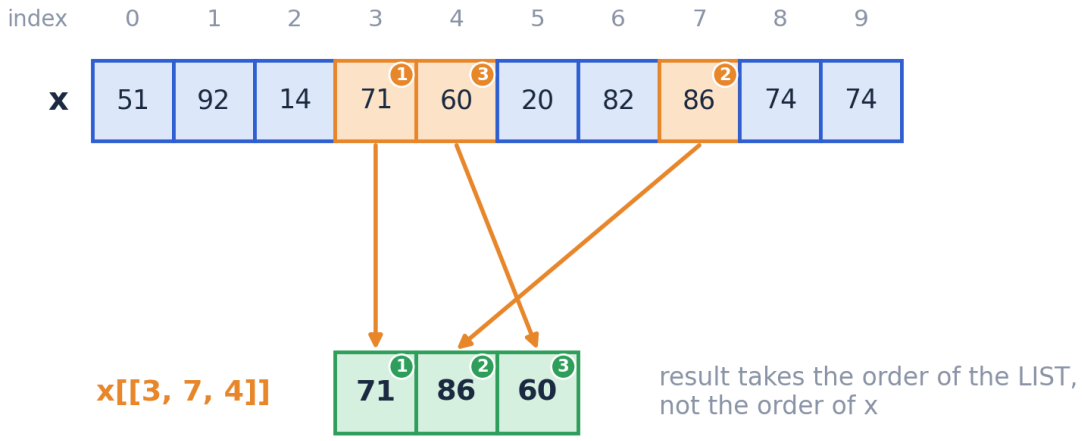
```
import numpy as np
x = np.array([51, 92, 14, 71, 60, 20, 82, 86, 74, 74])
print(x[3])            # scalar index
print(x[2:6])          # slice
print(x[[0, 3, 7]])    # fancy index: a LIST of positions
```

▶ OUTPUT

```
71
[14 71 60 20]
[51 71 86]
```

1.2 Fancy indexing on a 1D array

Put a list (or array) of positions inside the brackets. Each number in the list is a position; the result holds the values at those positions, in **the order of your list**.



x[[3, 7, 4]] picks positions 3, 7, 4 (orange) and returns them in that order (green).

PYTHON • type this in Colab

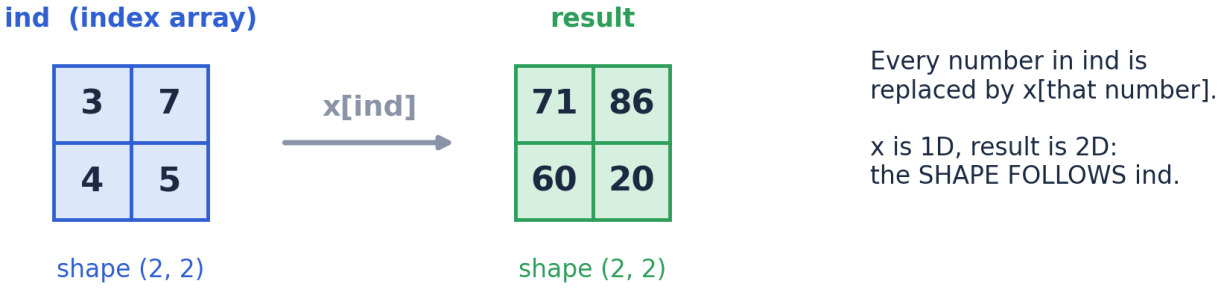
```
x = np.array([51, 92, 14, 71, 60, 20, 82, 86, 74, 74])
print(x[[3, 7, 4]])     # positions 3, 7, 4
print(x[[4, 4, 0]])     # repeats and any order are fine
```

▶ OUTPUT

```
[71 86 60]
[60 60 51]
```

The rule that matters most: the result takes the shape of the INDEX

If the index array is 2D, the result is 2D, even though x itself is 1D. NumPy simply replaces every number in the index array by the value at that position.



x is 1D with shape (10,), the index is 2×2, so the result is 2×2.

PYTHON • type this in Colab

```
x = np.array([51, 92, 14, 71, 60, 20, 82, 86, 74, 74])
ind = np.array([[3, 7],
                [4, 5]])
print(x[ind])
print(x[ind].shape)
```

▶ OUTPUT

```
[[71 86]
 [60 20]]
(2, 2)
```

MEMORY TRICK

Fancy indexing = **"stamp the index array with values"**. Whatever shape the index has, the answer has the same shape.

COMMON TRAP

Slicing gives a view, fancy indexing gives a copy. Changing the result of a slice changes the original array. Changing the result of a fancy index does **not** touch the original. NumPy never warns you about this.

PYTHON • type this in Colab

```
x = np.array([10, 20, 30, 40])
a = x[1:3]          # slice  -> view (shares memory with x)
b = x[[1, 2]]       # fancy  -> copy (independent)
a[0] = 999          # changes x
b[1] = -1           # does NOT change x
print(x)
```

▶ OUTPUT

```
[ 10 999  30  40]
```

WHY IT MATTERS

This is how NLP models look up word vectors. A sentence becomes an array of token IDs, and the embedding table has one row per word. `embedding_table[token_ids]` is exactly fancy indexing: 4 token IDs in, 4 rows out. You will see this line inside every language model.

PYTHON • type this in Colab

```
emb = np.array([[0.1, 0.2],     # vector of word 0
                [0.5, 0.6],     # vector of word 1
                [0.9, 0.8]])    # vector of word 2
token_ids = np.array([2, 0, 2, 1])   # a 4-word sentence
print(emb[token_ids])
print(emb[token_ids].shape)
```

▶ OUTPUT

```
[[0.9 0.8]
 [0.1 0.2]
 [0.9 0.8]
 [0.5 0.6]]
(4, 2)
```

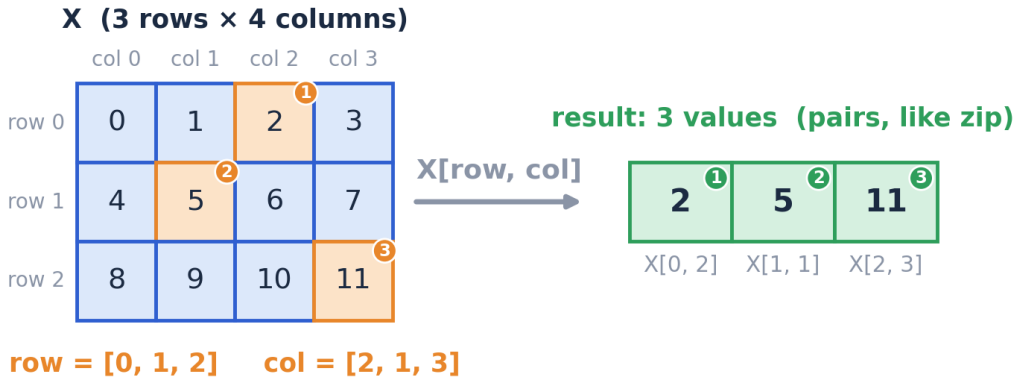
Word 2 appears twice in the sentence, so its row appears twice in the result. Repeats are allowed, which a slice could never do.

1.3 Fancy indexing in 2D: pairs

In 2D you give **two** index arrays: the first for rows, the second for columns. Here is the thing to understand before any code:

BIG PICTURE

The two arrays are **matched position by position**, exactly like Python's `zip`. Position 0 of `row` and position 0 of `col` form one address `(row, col)`. Position 1 forms the next address, and so on. 3 addresses in, **3 values out**, not a 3×3 table.



row = [0, 1, 2] and col = [2, 1, 3] give the three addresses (0,2), (1,1), (2,3).

PYTHON • type this in Colab

```
X = np.arange(12).reshape(3, 4)
row = np.array([0, 1, 2])
col = np.array([2, 1, 3])
print(X)
print(X[row, col])     # pairs: (0,2) (1,1) (2,3)
```

▶ OUTPUT

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[ 2  5 11]
```

MEMORY TRICK

`X[row, col]` = **zip(row, col)**. Same length in, same length out.

WHY IT MATTERS

The probability the model gave to the correct answer. A model outputs a probability table, one row per sample and one column per class. The loss function needs, for every sample, the probability of its **true** class. That is a pairing: sample 0 with its label, sample 1 with its label, and so on.

PYTHON • type this in Colab

```
probs = np.array([[0.7, 0.2, 0.1],     # sample 0
                  [0.1, 0.8, 0.1],     # sample 1
                  [0.3, 0.3, 0.4]])    # sample 2
labels = np.array([0, 1, 2])           # the correct class of each sample
print(probs[np.arange(3), labels])
```

▶ OUTPUT

```
[0.7 0.8 0.4]
```

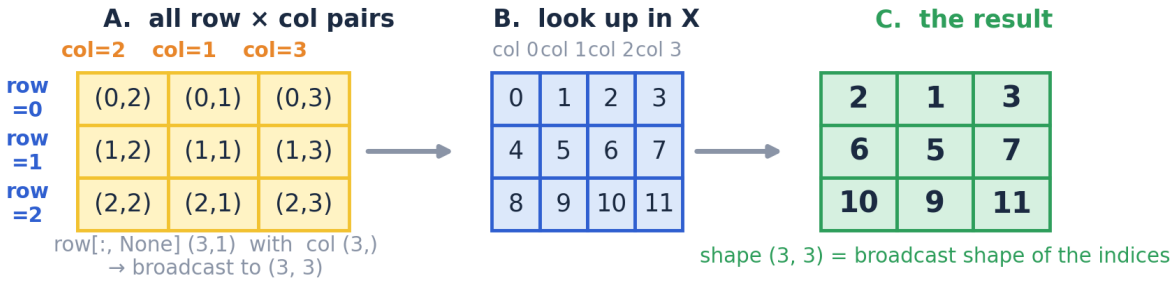
`np.arange(3)` is `[0, 1, 2]`, the row numbers. Paired with `labels` it picks `probs[0,0]`, `probs[1,1]`, `probs[2,2]`. This one line is the heart of the cross-entropy loss.

1.4 Fancy indexing in 2D: the grid

BIG PICTURE

Sometimes you do **not** want pairs. You want **every chosen row combined with every chosen column**, a small sub-table. You already know the tool for this from the broadcasting notes: make the row index a **column vector**. Then every row meets every column, just like a multiplication table.

Think of it in three stages (see the picture): **A.** NumPy builds a grid of addresses by broadcasting the (3,1) row index against the (3,) column index. **B.** It looks each address up in X. **C.** The result has the broadcast shape (3, 3), **not** the shape of X.



X[row[:, np.newaxis], col]: every row index meets every column index.

PYTHON • type this in Colab

```
X = np.arange(12).reshape(3, 4)
row = np.array([0, 1, 2])
col = np.array([2, 1, 3])
print(row[:, np.newaxis].shape, col.shape)
print(X[row[:, np.newaxis], col])
```

▶ OUTPUT

```
(3, 1) (3,)
[[ 2  1  3]
 [ 6  5  7]
 [10  9 11]]
```

Same idea, smaller, so you can check it by eye. We want rows 0 and 2 and columns 1 and 3:

PYTHON • type this in Colab

```
X = np.arange(12).reshape(3, 4)
rows = np.array([0, 2])
cols = np.array([1, 3])
print(X[rows, cols])                  # pairs  : (0,1) and (2,3)
print(X[rows[:, np.newaxis], cols])  # grid   : 2 rows x 2 columns
```

▶ OUTPUT

```
[ 1 11]
[[ 1  3]
 [ 9 11]]
```

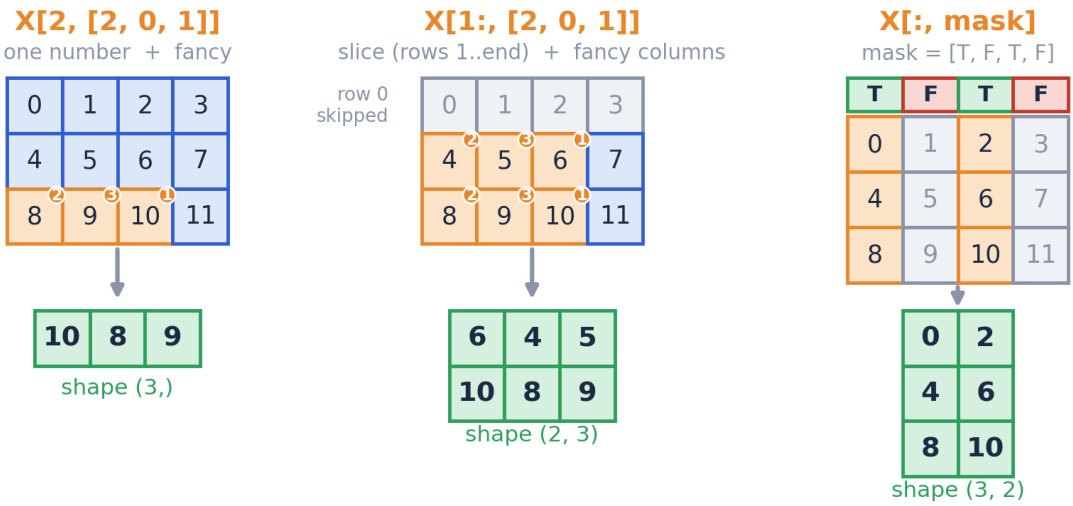
MEMORY TRICK

Plain row array and col array → **pairs** (a list). **Column-vector** row array and plain col array → **grid** (a table).

Tool	What it does	Priority
`X[row, col]`	pairs, like zip; result is 1D	MUST KNOW
`X[row[:, None], col]`	every row with every column; result is a grid	MUST KNOW
`X[np.ix_(rows, cols)]`	builds the same grid for you; handy shortcut	LOOK UP

1.5 Combined indexing

Fancy indexing mixes freely with the other styles you know. The usual pattern: **slice or pick one axis, fancy-index the other.**



Three combinations on the same 3×4 array X. Orange numbers show the order in which columns are picked.

PYTHON • type this in Colab

```
X = np.arange(12).reshape(3, 4)
print(X[2, [2, 0, 1]])          # one number + fancy
print(X[1:, [2, 0, 1]])         # slice + fancy
mask = np.array([True, False, True, False])
print(X[:, mask])               # all rows, columns where mask is True
```

▶ OUTPUT

```
[10  8  9]
[[ 6  4  5]
 [10  8  9]]
[[ 0  2]
 [ 4  6]
 [ 8 10]]
```

WHY IT MATTERS

A dataset is a 2D array: rows are samples, columns are features. `data[:, [0, 3]]` means "all samples, only features 0 and 3". That is **feature selection**. `X[train_idx]` selects the training samples. This combination is how you cut a dataset both ways.

PYTHON • type this in Colab

```
data = np.arange(20).reshape(4, 5)      # 4 samples, 5 features
print(data)
print(data[:, [0, 3]])                  # keep only feature 0 and feature 3
```

▶ OUTPUT

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
[[ 0  3]
 [ 5  8]
 [10 13]
 [15 18]]
```

1.6 Integer index versus Boolean mask (the trap)

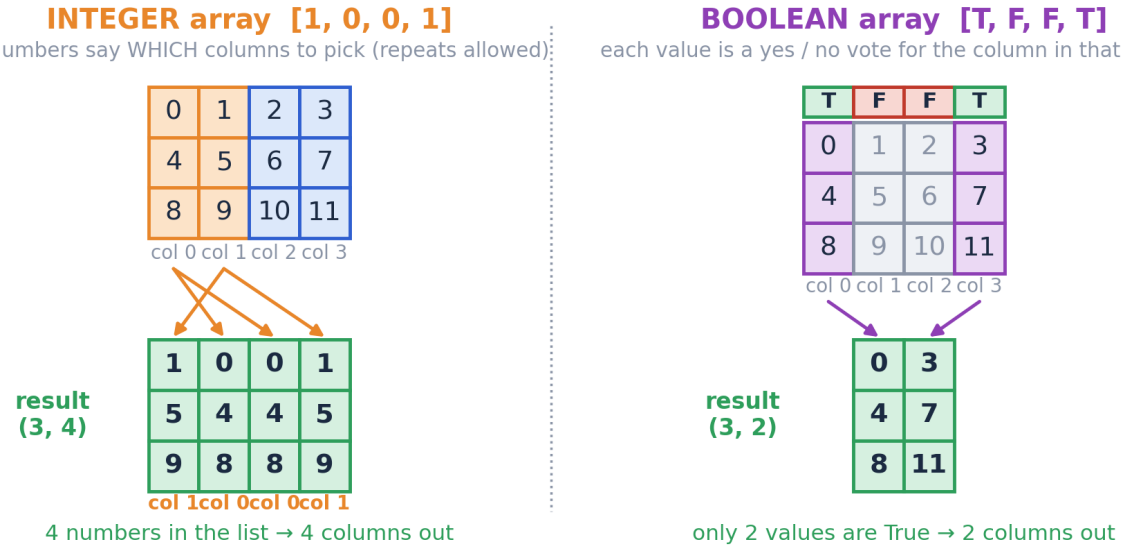
COMMON TRAP

An index array of **integers** and an index array of **Booleans** look almost the same but behave completely differently. Reading `[False, True, True, False]` as the numbers `[0, 1, 1, 0]` is a very common mistake. **The dtype decides what the array means.**

BIG PICTURE

Integer array: the numbers are positions. "Pick column 1, then column 0, then column 0, then column 1." The result has as many columns as the list has entries.

Boolean array: the values are yes/no votes. Position 0 of the mask votes on column 0, position 1 votes on column 1, and so on. Only the columns that voted True survive, so the result has as many columns as there are `True`s.



Same shape of index array, completely different meaning. Left: integers. Right: Booleans.

PYTHON • type this in Colab

```
X = np.arange(12).reshape(3, 4)
a = X[:, np.array([1, 0, 0, 1])]                    # integers
b = X[:, np.array([1, 0, 0, 1], dtype=bool)]        # Booleans
print(a)
print(a.shape)
print(b)
print(b.shape)
```

▶ OUTPUT

```
[[1 0 0 1]
 [5 4 4 5]
 [9 8 8 9]]
(3, 4)
[[ 0  3]
 [ 4  7]
 [ 8 11]]
(3, 2)
```

In `a`, every row uses the same column list `[1, 0, 0, 1]`, so row 1 `[4, 5, 6, 7]` becomes `[5, 4, 4, 5]`. When unsure, write out **one row** and apply the list to it, then repeat for the other rows.

The book's mask example, one step at a time

The book writes `X[row[:, np.newaxis], mask]`. Break it into three small steps:

PYTHON • type this in Colab

```
X = np.arange(12).reshape(3, 4)
row = np.array([0, 1, 2])
mask = np.array([1, 0, 1, 0], dtype=bool)
print(mask)                              # Step 1: the mask is [True, False, True, False]
print(X[row[:, np.newaxis], mask])       # Step 2: rows 0,1,2 (as a grid) with columns 0 and 2
print(X[:, mask])                        # Step 3: the simpler way to write the same thing
```

▶ OUTPUT

```
[ True False  True False]
[[ 0  2]
 [ 4  6]
 [ 8 10]]
[[ 0  2]
 [ 4  6]
 [ 8 10]]
```

Step 1: a True at positions 0 and 2 means "keep columns 0 and 2". Step 2: the (3,1) row index meets the two kept columns, giving a 3×2 grid. Step 3: `X[:, mask]` says "all rows, columns where the mask is True" and gives the identical result. In practice you will almost always write the Step 3 form.

WHY IT MATTERS

In real code the mask comes from a **condition**, not from typing it by hand. Example: keep only the features that actually vary, because a column with no variation tells a model nothing.

PYTHON • type this in Colab

```
data = np.array([[1., 5., 3.0],
                 [1., 7., 3.2],
                 [1., 9., 2.8]])
keep = data.var(axis=0) > 0.1      # one True/False per feature (column)
print(keep)
print(data[:, keep])
```

▶ OUTPUT

```
[False  True False]
[[5.]
 [7.]
 [9.]]
```

1.7 Selecting random points and splitting data

BIG PICTURE

Choosing random samples from a dataset is a two-step job. **Step 1:** make a list of random row numbers (no repeats). **Step 2:** use that list as a fancy index. That is all. The same idea is behind every train/test split and every mini-batch in deep learning.

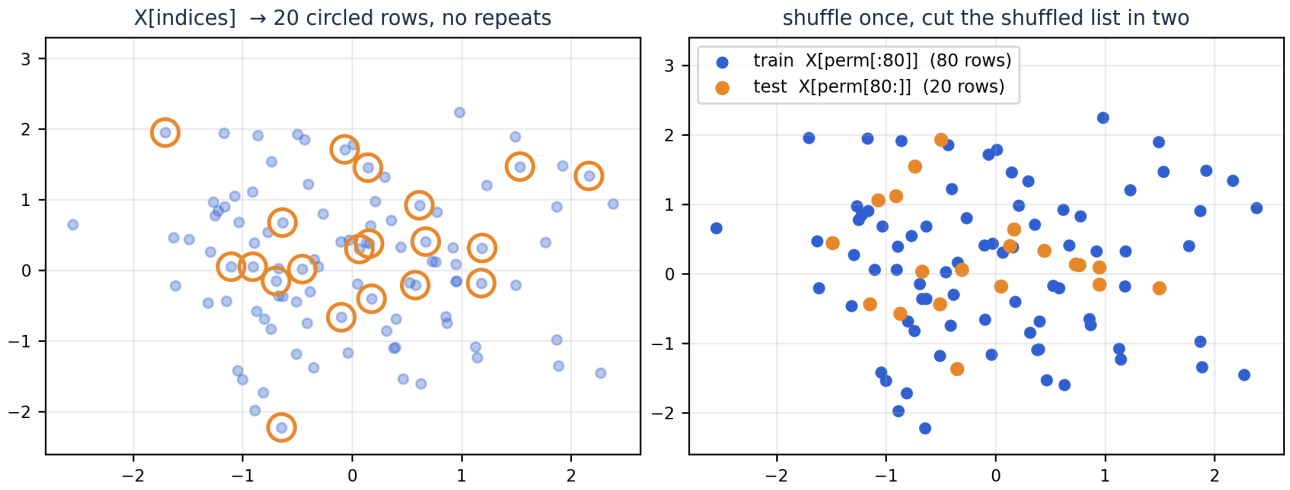
PYTHON • type this in Colab

```
np.random.seed(0)
X = np.random.randn(100, 2)                      # 100 points, 2 features each
indices = np.random.choice(100, 20, replace=False)
selection = X[indices]                           # fancy indexing picks 20 rows
print(indices)
print(selection.shape)
```

▶ OUTPUT

```
[15 91 28 93 89 14 32 67 43 21 84 34 64 77 99  5 75 59 68 62]
(20, 2)
```

`np.random.choice(100, 20, replace=False)` draws 20 numbers from 0..99. `replace=False` means a number cannot be drawn twice, so you get 20 **different** rows. The result is `(20, 2)`: 20 rows, and every row keeps both of its columns.



Left: 20 randomly chosen rows circled. Right: a train/test split made from one shuffled list.

A train/test split in three lines

PYTHON • type this in Colab

```
np.random.seed(0)
X = np.random.randn(100, 2)
perm = np.random.permutation(100)               # 0..99 in random order, each once
train, test = X[perm[:80]], X[perm[80:]]
print(train.shape, test.shape)
print(len(set(perm[:80]) & set(perm[80:])))     # rows that are in BOTH sets
```

▶ OUTPUT

```
(80, 2) (20, 2)
0
```

IMPORTANT

`replace=False` stops a repeat **inside one draw**. It does **not** protect two separate draws from overlapping. A split must take both sets from **one** shuffled list (`perm`), so every row is used exactly once and the overlap is 0. A row in both sets is called **data leakage**, and it makes your model score look better than it really is.

PYTHON • type this in Colab

```
np.random.seed(1)
print(np.random.choice(5, 8, replace=True))     # bootstrap: repeats ARE allowed
```

▶ OUTPUT

```
[3 4 0 1 3 0 0 1]
```

With `replace=True` the same row can be drawn again. That is **bootstrap sampling**, the idea behind random forests.

1.8 Modifying values with fancy indexing

Fancy indexing also works on the **left side** of an `=`, so you can change many positions at once. Every assignment operator works: `=`, `+=`, `-=`, `*=` and so on.

PYTHON • type this in Colab

```
x = np.arange(10)
i = np.array([2, 1, 8, 4])
x[i] = 99
print(x)
x[i] -= 10
print(x)
```

▶ OUTPUT

```
[ 0 99 99  3 99  5  6  7 99  9]
[ 0 89 89  3 89  5  6  7 89  9]
```

Repeated indices: the trap

When the same position appears twice, the **last** assignment wins. That part is easy to accept:

PYTHON • type this in Colab

```
x = np.zeros(10)
x[[0, 0]] = [4, 6]      # x[0] = 4, then x[0] = 6
print(x[0])
```

▶ OUTPUT

```
6.0
```

The surprising one is `+=`. Position 3 appears twice and position 4 appears three times, so you would expect 2 and 3:

PYTHON • type this in Colab

```
x = np.zeros(6)
i = [2, 3, 3, 4, 4, 4]
x[i] += 1
print(x)
```

▶ OUTPUT

```
[0. 0. 1. 1. 1. 0.]
```

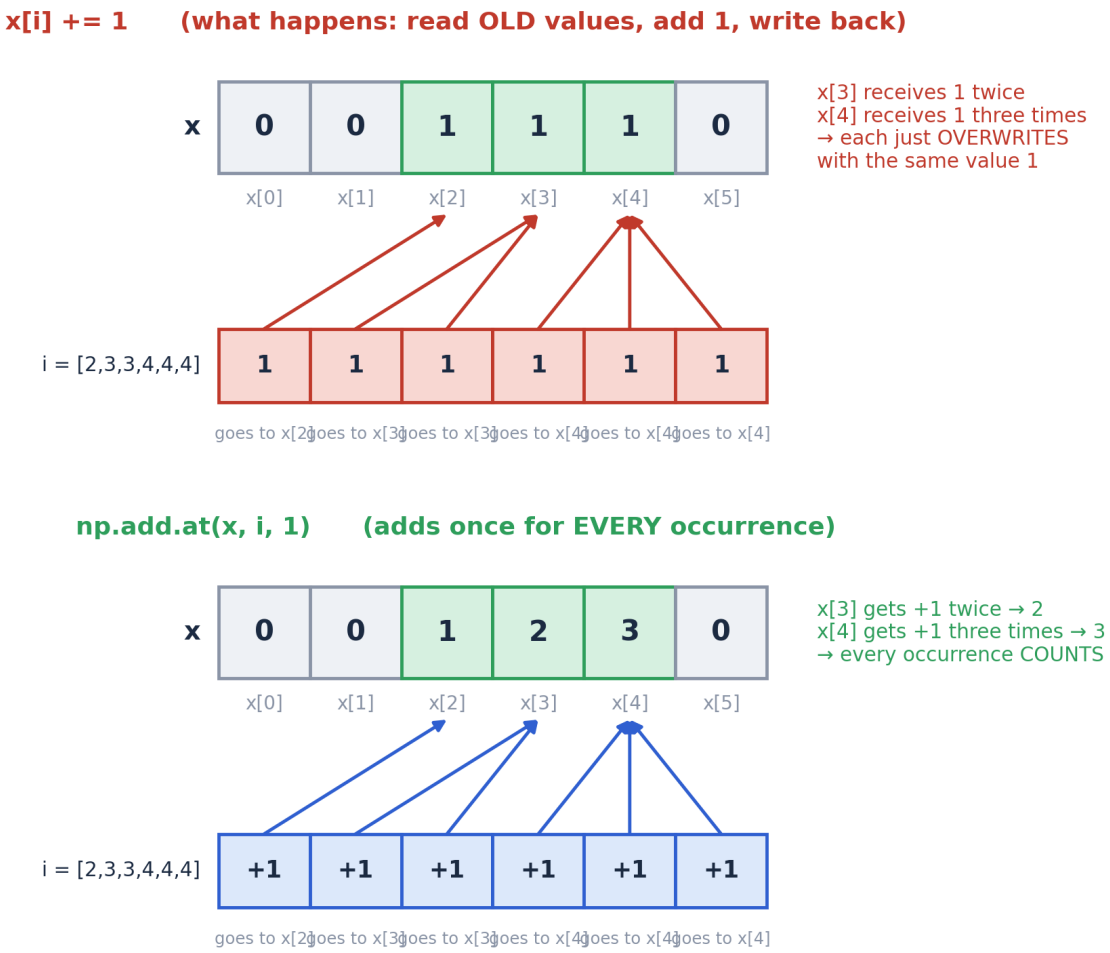
You get 1, 1, 1, not 1, 2, 3. The reason is that `x[i] += 1` is shorthand for `x[i] = x[i] + 1`, which happens in **two separate steps**. Doing those steps by hand makes it obvious:

PYTHON • type this in Colab

```
x = np.zeros(6)
i = [2, 3, 3, 4, 4, 4]
temp = x[i] + 1       # Step 1: read the OLD values (all 0) and add 1
print(temp)
x[i] = temp           # Step 2: write back. Position 4 is written 3 times, always with 1
print(x)
```

▶ OUTPUT

```
[1. 1. 1. 1. 1. 1.]
[0. 0. 1. 1. 1. 0.]
```



Top: x[i] += 1 writes the same 1 repeatedly. Bottom: np.add.at adds once for every occurrence.

The addition happened once; it was the **writing** that was repeated. To make every occurrence count, use `np.add.at`:

PYTHON • type this in Colab

```
x = np.zeros(6)
i = [2, 3, 3, 4, 4, 4]
np.add.at(x, i, 1)      # in-place: adds 1 for EVERY occurrence in i
print(x)
```

▶ OUTPUT

```
[0. 0. 1. 2. 3. 0.]
```

WHY IT MATTERS

Counting. How many times does each word ID appear in a corpus, or how many samples fall in each class? That is `np.add.at(counts, word_ids, 1)`. For plain counting, `np.bincount` does the same in one call. PyTorch has the same trap in `index_add_`, and the same logic is why gradients of a repeated word must be **added** (a word used 3 times needs 3 contributions).

PYTHON • type this in Colab

```
word_ids = np.array([2, 0, 2, 2, 1])
counts = np.zeros(3)
np.add.at(counts, word_ids, 1)
print(counts)
print(np.bincount(word_ids))      # the one-call shortcut for plain counting
```

▶ OUTPUT

```
[1. 1. 3.]
[1 1 3]
```

MEMORY TRICK

`x[i] += 1` = **"overwrite"** (repeats collapse). `np.add.at(x, i, 1)` = **"accumulate"** (repeats count).

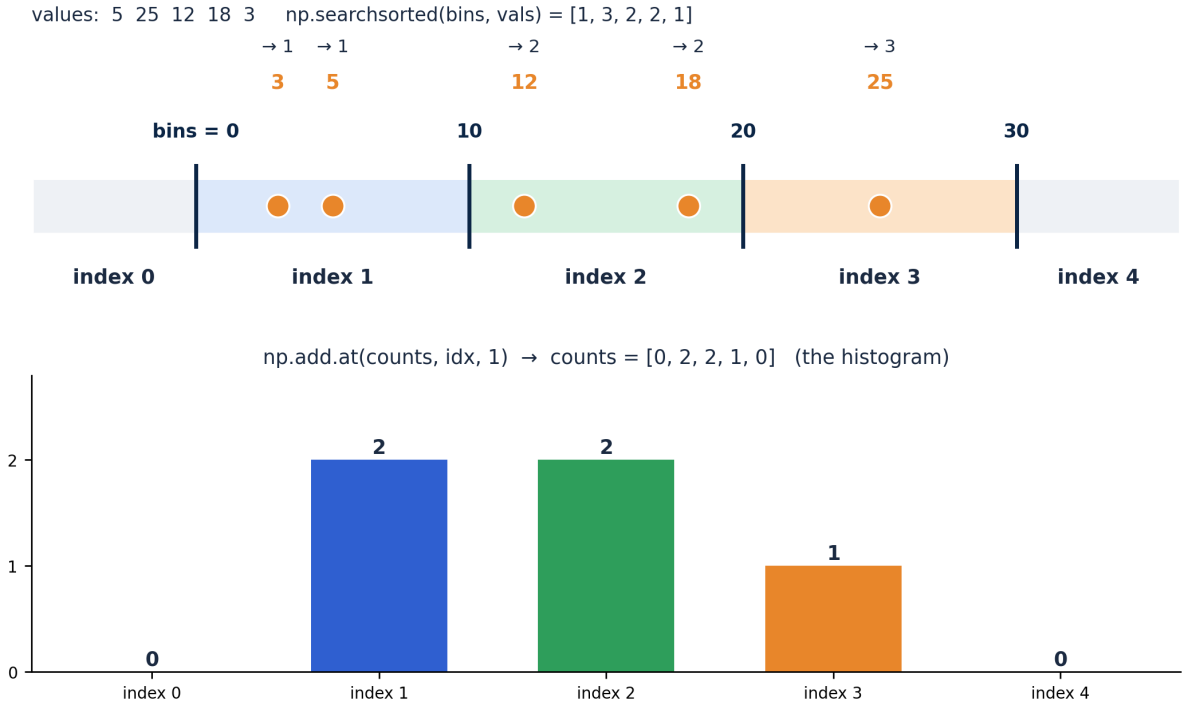
1.9 Binning data with searchsorted

BIG PICTURE

A **histogram** sorts values into buckets and counts how many landed in each bucket. This takes two steps: **(1)** for every value, find **which bucket** it belongs to. **(2)** count the values per bucket. The new tool is `np.searchsorted` for step 1; step 2 is `np.add.at` from the last topic.

ANALOGY

Exam marks into grades. Cut-offs 40, 60, 75, 90 divide the marks into five grade buckets. For each student you ask "which bucket does my mark fall in?" That question is what `searchsorted` answers, for all students at once.



Each value gets a bucket index from searchsorted; add.at then counts the values per bucket.

PYTHON • type this in Colab

```
bins = np.array([0, 10, 20, 30])
vals = np.array([5, 25, 12, 18, 3])
i = np.searchsorted(bins, vals)        # bucket index of every value
print(i)
counts = np.zeros(4)
np.add.at(counts, i, 1)                # count how many landed in each bucket index
print(counts)
```

▶ OUTPUT

```
[1 3 2 2 1]
[0. 2. 2. 1.]
```

Reading `i`: 5 and 3 sit between 0 and 10, so both get index 1. 12 and 18 sit between 10 and 20, so both get index 2. 25 sits between 20 and 30, so it gets index 3. `np.add.at` then counts them. Without `add.at`, repeated indices (1 and 2 appear twice) would be counted only once, which is the trap from the last topic.

The same trick turns marks into grade letters, because the bucket index can itself be used as a **fancy index**:

PYTHON • type this in Colab

```
cutoffs = np.array([40, 60, 75, 90])
marks   = np.array([35, 62, 80, 95, 55])
idx = np.searchsorted(cutoffs, marks)
grades = np.array(['F', 'D', 'C', 'B', 'A'])
print(idx)
print(grades[idx])
```

▶ OUTPUT

```
[0 2 3 4 1]
['F' 'C' 'B' 'A' 'D']
```

The book's histogram by hand

PYTHON • type this in Colab

```
np.random.seed(42)
x = np.random.randn(100)
bins = np.linspace(-5, 5, 20)          # 20 bin edges from -5 to 5
counts = np.zeros_like(bins)
i = np.searchsorted(bins, x)
np.add.at(counts, i, 1)
print(counts)
print(counts.sum())                    # all 100 values were counted
```

▶ OUTPUT

```
[ 0.  0.  0.  0.  0.  1.  3.  7.  9. 23. 22. 17. 10.  7.  1.  0.  0.  0.
  0.  0.]
100.0
```

VERSION NOTE

The book plots this with `plt.plot(bins, counts, linestyle='steps')`. Recent Matplotlib versions raise an error for `linestyle='steps'`. Write `plt.plot(bins, counts, drawstyle='steps')` instead. In real work you would simply call `plt.hist(x, bins)`.

NOTE

The book's speed lesson (we did not rerun the timings): the hand-made search-and-count was faster than `np.histogram` for 100 points, but `np.histogram` won clearly for 1,000,000 points. The lesson is that **algorithm speed depends on data size**, and that built-in NumPy routines are tuned for big data. Know the building blocks, but use the built-in `np.histogram` or `plt.hist` for real work.

PART 2

2 Sorting Arrays

Block 8: using sort, argsort and partition as tools on data

IN THIS PART YOU WILL LEARN

- ✓ What Block 8 is (and is not): **NumPy sorting as a tool**, not DSA sorting algorithms
- ✓ `np.sort` versus `x.sort()`, and the `None` trap
- ✓ `np.argsort`: the **order** instead of the sorted values (ranking, top-k)
- ✓ Sorting **along an axis**, and why it can silently break your rows
- ✓ `np.partition` and `np.argpartition`: the k smallest without a full sort

2.1 What Block 8 is, and what it is not

BIG PICTURE

You heard that "sorting questions" are asked in DSA. That is true, but **this block is not about them**. DSA sorting asks "can you write bubble sort or merge sort by hand?" This block asks "NumPy already sorts for you very fast, so how do you use it on data?" The book opens with selection sort and bogosort only as a warm-up to show how slow naive sorting is, then moves on to the built-in tools.

	DSA sorting questions	Block 8 (NumPy sorting)
Goal	Write the algorithm yourself	Use the fast built-in tools
Typical task	Implement bubble / merge / quick sort	Rank scores, find top-k, order rows
Where it is asked	Placements, coding rounds	Every ML script you will write
In these notes	Not covered (study it separately later)	Fully covered

NOTE

Skipped on purpose: the `selection_sort` and `bogosort` code from the book. The only idea worth keeping is the scaling comparison: selection sort needs about N^2 steps, bogosort needs about $N \times N!$ steps (hopeless), and NumPy's `np.sort` needs about N log N steps. We explain this scaling properly in **Part 4 (Big-O)**.

2.2 np.sort and x.sort()

Two ways to sort, with one difference that causes real bugs:

Style	What it does	Priority
`np.sort(x)`	returns a **new** sorted array, `x` is untouched	MUST KNOW
`x.sort()`	sorts `x` **in place** and returns nothing (`None`)	MUST KNOW

PYTHON • type this in Colab

```
x = np.array([2, 1, 4, 3, 5])
print(np.sort(x))        # new sorted copy
print(x)                 # original unchanged
x.sort()                 # now x itself is sorted
print(x)
```

▶ OUTPUT

```
[1 2 3 4 5]
[2 1 4 3 5]
[1 2 3 4 5]
```

COMMON TRAP `x = x.sort()` throws your data away. In-place methods return `None`, so `x` becomes `None`.

PYTHON • type this in Colab

```
y = np.array([3, 1, 2])
y = y.sort()
print(y)
```

▶ OUTPUT

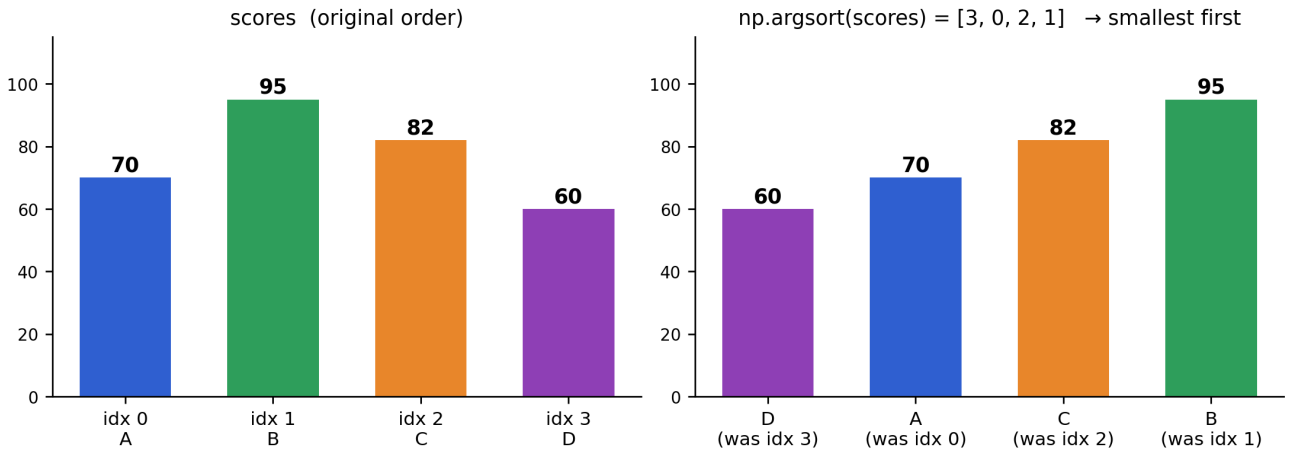
```
None
```

By default NumPy uses a fast quicksort-style method (about N log N steps). `mergesort` and `heapsort` are available but the default is fine for almost everything.

2.3 np.argsort: the order, not the values

BIG PICTURE `np.sort` gives you the **sorted values**. `np.argsort` gives you **where each value came from**: the positions, in sorted order. The first number is the position of the smallest value, the second is the position of the second smallest, and so on.

ANALOGY **A race.** `np.sort` gives the finishing **times**: 58, 61, 63. `np.argsort` gives **who** finished first, second, third: runner 3, runner 0, runner 2. With the runner numbers you can look up anything else about them (name, age, team), not just the time.



argsort returns the positions that would sort the array: the smallest 60 was at index 3, the next 70 at index 0, and so on.

PYTHON • type this in Colab

```
x = np.array([2, 1, 4, 3, 5])
i = np.argsort(x)
print(i)          # positions of the smallest, 2nd smallest, ...
print(x[i])       # fancy indexing (Part 1) rebuilds the sorted array
```

▶ OUTPUT

```
[1 0 3 2 4]
[1 2 3 4 5]
```

The power of `argsort` is that the same positions can reorder **other** arrays that go with the scores:

PYTHON • type this in Colab

```
names  = np.array(['A', 'B', 'C', 'D'])
scores = np.array([70, 95, 82, 60])
order = np.argsort(scores)
print(order)
print(names[order])             # names from lowest to highest score
print(names[order[::-1]])       # names from highest to lowest score
```

▶ OUTPUT

```
[3 0 2 1]
['D' 'A' 'C' 'B']
['B' 'C' 'A' 'D']
```

WHY IT MATTERS

Top-k predictions. A language model gives a probability for every word in its vocabulary. The 5 most likely next words are `np.argsort(-probs)[:5]`. The minus sign flips the order so the biggest comes first. Ranking search results works the same way.

PYTHON • type this in Colab

```
probs = np.array([0.05, 0.40, 0.10, 0.30, 0.15])    # probability of 5 words
top3 = np.argsort(-probs)[:3]                        # positions of the 3 biggest
print(top3)
print(probs[top3])
```

▶ OUTPUT

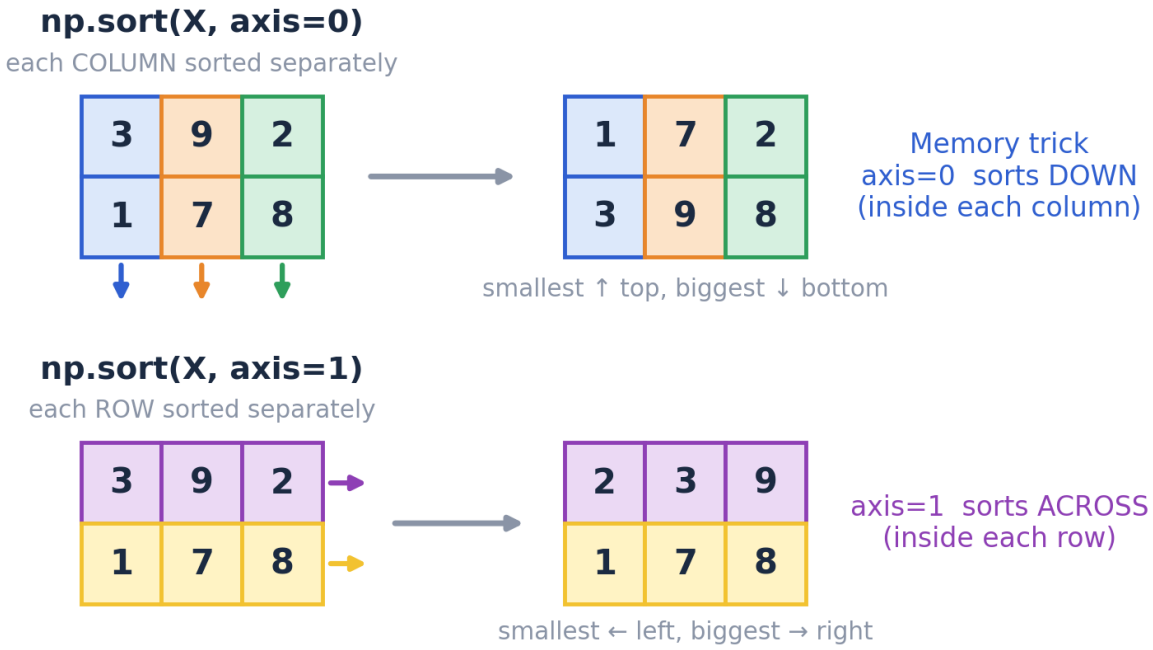
```
[1 3 4]
[0.4  0.3  0.15]
```

MEMORY TRICK

argsort = "arg" for argument = position. It answers "which positions, in what order?", and you use the answer as a fancy index.

2.4 Sorting along an axis

For a 2D array, the `axis` argument says **in which direction** each sort runs. Every row (or every column) is treated as its own separate little array.



axis=0 sorts inside every column (colours stay in their column). axis=1 sorts inside every row.

PYTHON • type this in Colab

```
X = np.array([[3, 9, 2],
              [1, 7, 8]])
print(np.sort(X, axis=0))     # each COLUMN sorted
print(np.sort(X, axis=1))     # each ROW sorted
```

▶ OUTPUT

```
[[1 7 2]
 [3 9 8]]
[[2 3 9]
 [1 7 8]]
```

MEMORY TRICK

axis=0 sorts DOWN (inside each column). **axis=1 sorts ACROSS** (inside each row).

COMMON TRAP

Sorting an axis destroys the link between values that belong together. Each row or column is sorted on its own, so a row is no longer the same sample. Below, each row is a pair (key, value). Sorting `axis=0` sorts keys and values separately and glues wrong pairs together.

PYTHON • type this in Colab

```
data = np.array([[3, 10],
                 [1, 30],
                 [2, 20]])              # (key, value) pairs
print(np.sort(data, axis=0))            # WRONG: keys and values sorted separately
order = np.argsort(data[:, 0])          # order of the keys only
print(data[order])                      # RIGHT: whole rows move together
```

▶ OUTPUT

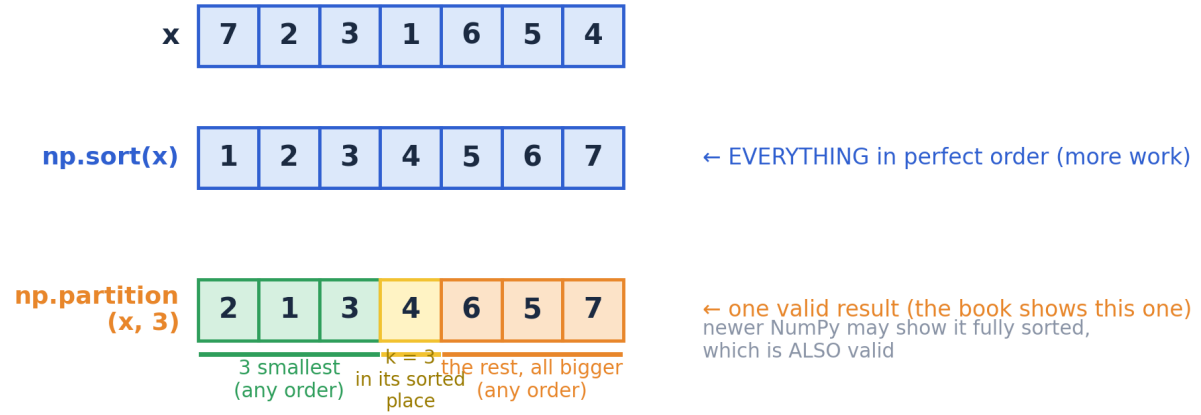
```
[[ 1 10]
 [ 2 20]
 [ 3 30]]
[[ 1 30]
 [ 2 20]
 [ 3 10]]
```

In the first output the pair `(1, 30)` became `(1, 10)`: the data was silently corrupted. The second way sorts **only the key column** with `argsort` and then moves **whole rows** with fancy indexing. Use that pattern whenever rows must stay intact.

2.5 Partial sorts: partition and argpartition

BIG PICTURE

Sometimes you do not need everything in order. You only need **the k smallest** (or largest). Fully sorting a million items just to read the first 5 is wasted work. `np.partition(x, k)` does the cheaper job: it makes sure the k smallest values are on the **left** and the rest are on the **right**, but it does **not** sort inside either side.



partition(x, 3): three smallest on the left, the k-th element in its sorted place, the rest on the right. Inside each side the order is free.

PYTHON • type this in Colab

```
x = np.array([7, 2, 3, 1, 6, 5, 4])
print(np.partition(x, 3))
```

▶ OUTPUT

```
[1 2 3 4 5 6 7]
```

IMPORTANT

Why your Colab may show a fully sorted array. The book shows `[2 1 3 4 6 5 7]`. Newer NumPy versions often return `[1 2 3 4 5 6 7]` for small arrays. Both are valid. The only promise is: the k smallest are on the left, everything bigger is on the right, and the order **inside** the groups is unspecified. On tiny arrays the algorithm happens to finish sorted, which looks like a full sort but is not guaranteed. Never write code that relies on the inside order.

A bigger array shows the real behaviour. We check the promise (left side all smaller than right side) and show that the right side is **not** sorted:

PYTHON • type this in Colab

```
np.random.seed(0)
x = np.random.permutation(1000)
p = np.partition(x, 10)
print(np.sort(p[:10]))                          # the 10 smallest values are in the first 10 slots
print(p[:10].max() < p[10:].min())              # every left value is smaller than every right value
print(np.array_equal(p[10:], np.sort(p[10:])))  # is the right side sorted?  (not required to be)
```

▶ OUTPUT

```
[0 1 2 3 4 5 6 7 8 9]
True
False
```

For the **largest** values use a negative k. `np.partition(x, -3)` puts the 3 biggest at the end:

PYTHON • type this in Colab

```
x = np.array([7, 2, 3, 1, 6, 5, 4])
print(np.sort(np.partition(x, -3)[-3:]))   # the 3 largest values (sorted here only for display)
```

▶ OUTPUT

```
[5 6 7]
```

`np.argpartition` is to `partition` what `argsort` is to `sort`: it returns **positions** instead of values. This is the version you will use for top-k:

PYTHON • type this in Colab

```
scores = np.array([70, 95, 82, 60, 88])
top2 = np.argpartition(scores, -2)[-2:]          # positions of the 2 biggest (order not guaranteed)
print(np.sort(top2))
print(scores[top2].max(), scores[top2].min())
best_first = top2[np.argsort(-scores[top2])]      # sort just those 2 if you need them in order
print(best_first)
print(scores[best_first])
```

▶ OUTPUT

```
[1 4]
95 88
[1 4]
[95 88]
```

MEMORY TRICK **partition = "split into two camps"**, **sort = "full ranking"**. Use partition for top-k, then sort only the k winners if you need them in order.

WHY IT MATTERS A full sort costs about N log N steps, partition only about N. For top-k over millions of items (search results, vocabulary, nearest neighbours) that gap is large. Part 3 uses exactly this idea.

PART 3

3 k-Nearest Neighbours, from zero

The example that ties broadcasting, aggregation and sorting together

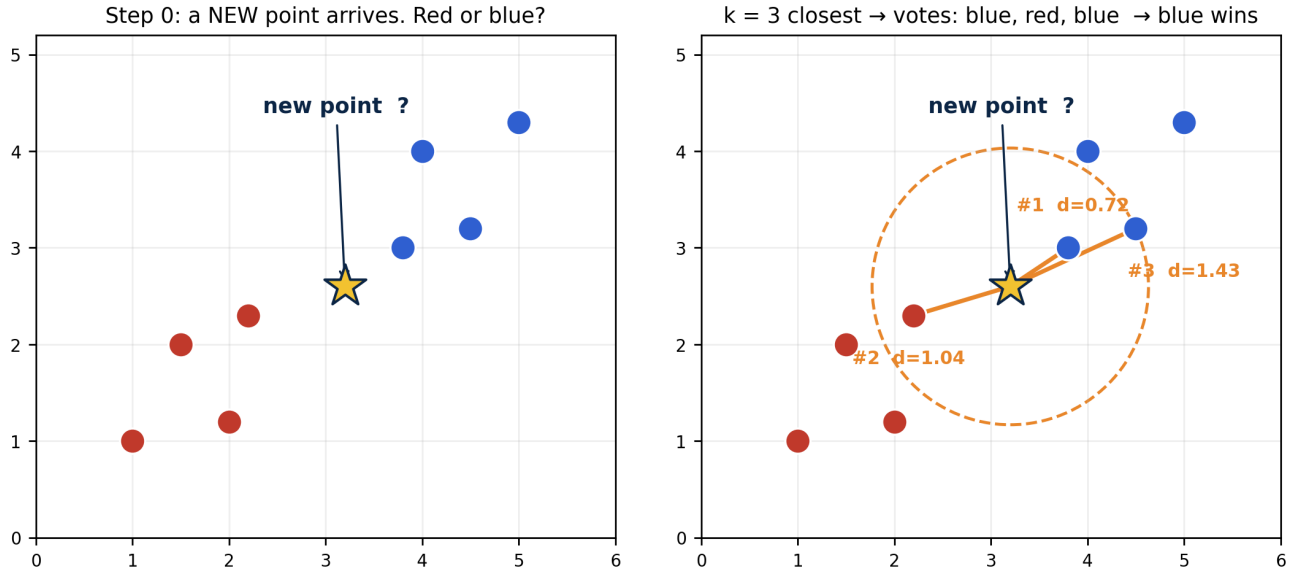
IN THIS PART YOU WILL LEARN

- ✓ The **big idea** of KNN with a real-life picture, before any code
- ✓ How to measure the **distance** between two points, then from one point to many
- ✓ How to build the whole **N × N distance table**, first with loops, then in **one broadcast line**
- ✓ How to read **argsort on each row** to get every point's neighbours
- ✓ `argpartition` for speed, a reusable function, and where this appears in real AI/ML (embeddings, RAG)

3.1 The big idea, with a real-life picture

ANALOGY

The new student. A new student walks into a class and you want to guess which friend group she will join. A simple method: look at the **3 students sitting closest to her** and see which group most of them belong to. If 2 of 3 belong to group Blue, guess Blue. That is the entire algorithm. It is called **k-Nearest Neighbours** (KNN), with k = 3 here.



Left: a new point arrives. Right: its 3 closest known points vote. Blue gets 2 votes, red gets 1, so the guess is blue.

BIG PICTURE

KNN has only four steps. Everything in this part is just doing these four steps with NumPy:

Step 1. Measure the distance from the new point to every known point.

Step 2. Sort the points by that distance (closest first).

Step 3. Take the first k of them.

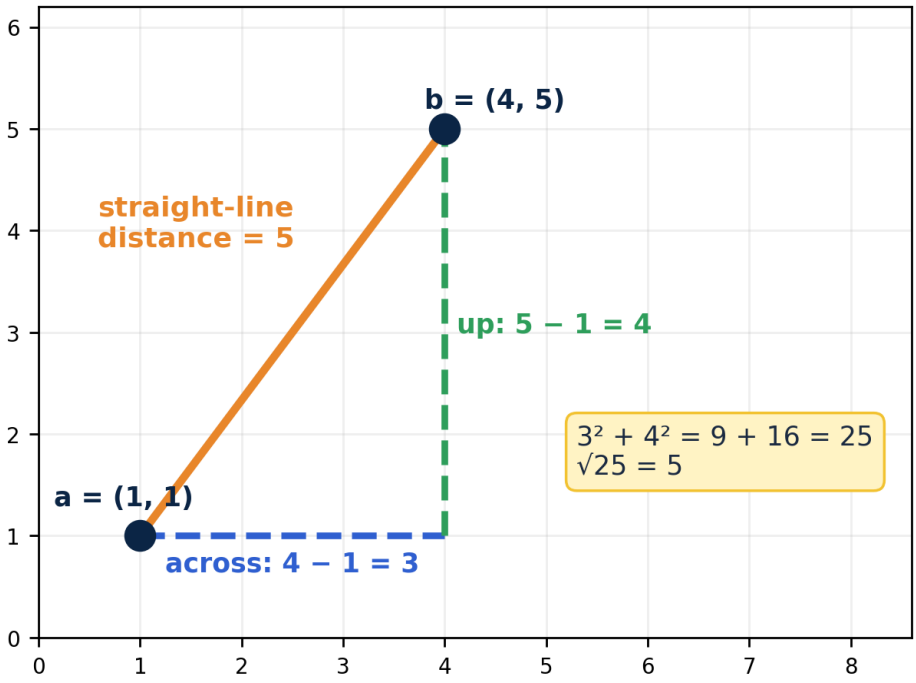
Step 4. Let those k points vote.

IMPORTANT

What the book's example actually does: it does Steps 1 to 3 for **every point at once** (it finds the neighbours of all 10 points in one go). It never does the vote. So the hard-looking line in the book is not "KNN"; it is the machinery that finds **who is close to whom**. We will build it slowly, from one pair of points up to the full table.

3.2 Step 1: the distance between two points

Take the points `a = (1, 1)` and `b = (4, 5)`. The straight-line distance comes from Pythagoras: go across by 3, go up by 4, and the straight line is the long side of a right-angled triangle.



distance² = across² + up² = 3² + 4² = 25, so the distance is 5.

PYTHON • type this in Colab

```
a = np.array([1, 1])
b = np.array([4, 5])
diff = a - b
print(diff)              # how far apart in each direction: [-3 -4]
sq = diff ** 2
print(sq)                # square each one: [9 16]
d_sq = sq.sum()
print(d_sq)              # add them up: 25  (the SQUARED distance)
print(np.sqrt(d_sq))     # the real distance: 5.0
```

▶ OUTPUT

```
[-3 -4]
[ 9 16]
25
5.0
```

IMPORTANT

We keep the distance squared and skip the square root. If one point is closer than another, its squared distance is also smaller, so the **ranking is identical**, and we save the work of a square root. KNN only cares about who is closer, not by how many units.

3.3 Step 2: the distance from ONE point to ALL points

Now 3 points, and we want the distance from point 0 to **each** of them (including itself). Subtracting `P[0]` from the whole array uses broadcasting you already know from Part 3: a (3,2) array minus a (2,) array. The single point is stretched down to every row.

PYTHON • type this in Colab

```
P = np.array([[0, 0],
              [3, 4],
              [1, 1]])
print(P - P[0])                          # every point minus point 0
print((P - P[0]) ** 2)                   # square every number
d = np.sum((P - P[0]) ** 2, axis=1)      # add the two numbers in each ROW
print(d)
```

▶ OUTPUT

```
[[0 0]
 [3 4]
 [1 1]]
[[ 0  0]
 [ 9 16]
 [ 1  1]]
[ 0 25  2]
```

Read the result `[0, 25, 2]`: point 0 is 0 away from itself, 25 away from point 1 (the (3,4) point, the 3-4-5 triangle from before), and 2 away from point 2. `axis=1` means "add across each row", which adds the x-part and the y-part of every point.

3.4 Steps 3 and 4: sort, take k, vote

Now we do the full KNN guess for the picture in 3.1. There are 8 known points, 4 red and 4 blue, and one new point.

PYTHON • type this in Colab

```
P = np.array([[1, 1], [1.5, 2], [2, 1.2], [2.2, 2.3],      # 4 red points
              [4, 4], [4.5, 3.2], [5, 4.3], [3.8, 3]])     # 4 blue points
labels = np.array(['red'] * 4 + ['blue'] * 4)
new = np.array([3.2, 2.6])

d = np.sum((P - new) ** 2, axis=1)       # Step 1: squared distance to every known point
k = 3
nearest = np.argsort(d)[:k]              # Steps 2 and 3: positions of the 3 closest
print(nearest)
print(labels[nearest])                   # their labels (fancy indexing again)

names, votes = np.unique(labels[nearest], return_counts=True)   # Step 4: count the votes
print(names, votes)
print(names[np.argmax(votes)])           # the winner
```

▶ OUTPUT

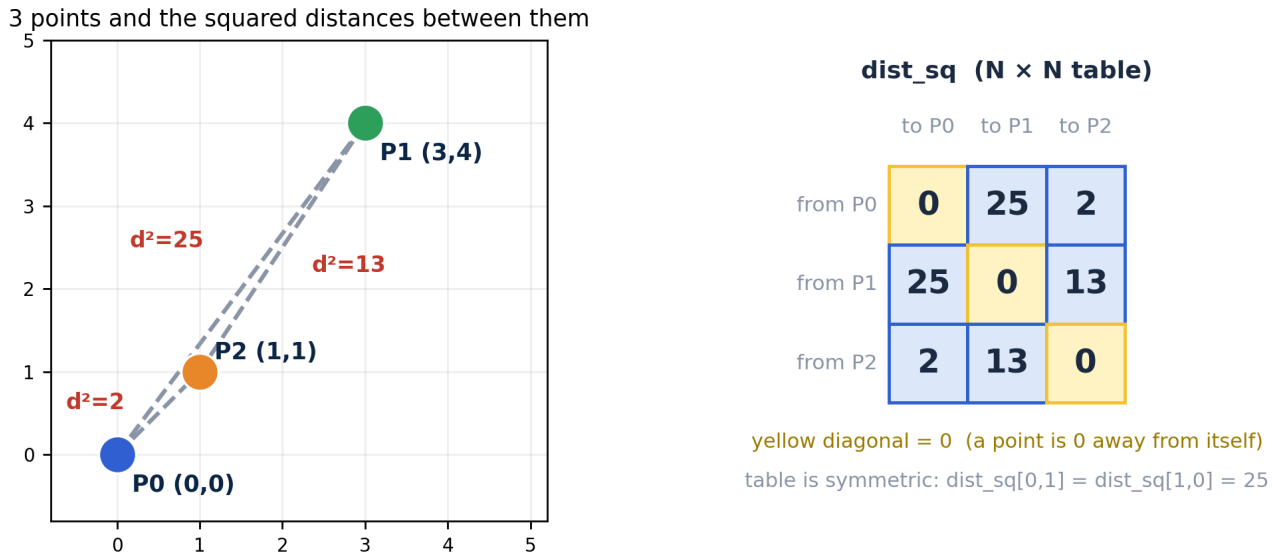
```
[7 3 5]
['blue' 'red' 'blue']
['blue' 'red'] [2 1]
blue
```

`argsort(d)[:k]` is the whole "find the k closest" step. `np.unique(..., return_counts=True)` counts how many times each label appears. The winner is the label with the biggest count.

3.5 Step 5: the distances between EVERY pair of points

BIG PICTURE

So far we asked "how far is **one** point from everyone?". The book asks it for **every** point at once. The answer is a **table**: row i, column j holds the squared distance between point i and point j. With N points the table has N × N entries.



Three points and their distance table. The diagonal is 0 and the table is symmetric.

The clearest way to build this table is with two loops. It is slow, but you can see exactly what is happening: for every pair (i, j), compute the squared distance.

PYTHON • type this in Colab

```
P = np.array([[0, 0], [3, 4], [1, 1]])
N = len(P)
dist_sq = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        dist_sq[i, j] = np.sum((P[i] - P[j]) ** 2)
print(dist_sq)
```

▶ OUTPUT

```
[[ 0. 25.  2.]
 [25.  0. 13.]
 [ 2. 13.  0.]]
```

This works, but loops in Python are very slow for large N. We want to produce the **same table** without loops.

3.6 Step 6: the one-line version, decoded

BIG PICTURE

This is the line that looked like "boom baam" in the book:

`dist_sq = np.sum((X[:, np.newaxis, :] - X[np.newaxis, :, :]) ** 2, axis=-1)`

It is **not new magic**. It is the "every row meets every column" grid trick from broadcasting (the multiplication-table idea), with **one extra twist**: each "number" in the grid is now a **pair** (x, y). So the grid needs one extra axis to hold the pair. We take it in 4 small moves.

Move 1: stretch the points both ways

`P[:, None, :]` makes the points into a **column** of shape (3,1,2), and `P[None, :, :]` makes the same points into a **row** of shape (1,3,2). Subtracting them stretches the column across and the row down, so **every point meets every point**. Cell [i, j] of the result is `P[i] - P[j]`.

A = P[:, None, :] shape (3,1,2)

(0,0)	(0,0)	(0,0)
(3,4)	(3,4)	(3,4)
(1,1)	(1,1)	(1,1)

copied ACROSS (to the right)

B = P[None, :, :] shape (1,3,2)

(0,0)	**(3,4)**	**(1,1)**
(0,0)	(3,4)	(1,1)
(0,0)	(3,4)	(1,1)

copied DOWN (to the bottom)

A − B shape (3,3,2)

(0,0)	**(-3,-4)**	**(-1,-1)**
(3,4)	**(0,0)**	**(2,3)**
(1,1)	**(-2,-3)**	**(0,0)**

cell [i, j] = P[i] − P[j]

The column A is copied across, the row B is copied down, and A − B subtracts matching cells.

PYTHON • type this in Colab

```
P = np.array([[0, 0], [3, 4], [1, 1]])
A = P[:, np.newaxis, :]       # shape (3, 1, 2)
B = P[np.newaxis, :, :]       # shape (1, 3, 2)
print(A.shape, B.shape)
diff = A - B                  # broadcasts to (3, 3, 2)
print(diff.shape)
print(diff)
```

▶ OUTPUT

```
(3, 1, 2) (1, 3, 2)
(3, 3, 2)
[[[ 0  0]
  [-3 -4]
  [-1 -1]]

 [[ 3  4]
  [ 0  0]
  [ 2  3]]

 [[ 1  1]
  [-2 -3]
  [ 0  0]]]
```

Reading `diff`: the top-left block `[[0 0], [-3 -4], [-1 -1]]` is point 0 minus point 0, minus point 1, minus point 2. There are 3 × 3 = 9 cells, and every cell holds a pair (dx, dy).

Moves 2 and 3: square it, then add the pair

1. diff (3,3,2)
subtract: A − B

(0, 0)	**(-3, -4)**	**(-1, -1)**
(3, 4)	**(0, 0)**	**(2, 3)**
(1, 1)	**(-2, -3)**	**(0, 0)**

each cell holds (dx, dy)

2. diff ** 2 (3,3,2)
square every number

(0, 0)	**(9, 16)**	**(1, 1)**
(9, 16)	**(0, 0)**	**(4, 9)**
(1, 1)	**(4, 9)**	**(0, 0)**

same shape, all numbers ≥ 0

3. .sum(axis=-1) (3,3)
add the 2 numbers in each cell

0	**25**	**2**
25	**0**	**13**
2	**13**	**0**

the 2 axes of D collapse into 1 number

The 3×3×2 block becomes the 3×3 distance table once the last axis is added up.

PYTHON • type this in Colab

```
P = np.array([[0, 0], [3, 4], [1, 1]])
diff = P[:, np.newaxis, :] - P[np.newaxis, :, :]
sq = diff ** 2                    # Move 2: square every number
print(sq.shape)
dist_sq = sq.sum(axis=-1)         # Move 3: add the 2 numbers inside every cell
print(dist_sq.shape)
print(dist_sq)
```

▶ OUTPUT

```
(3, 3, 2)
(3, 3)
[[ 0 25  2]
 [25  0 13]
 [ 2 13  0]]
```

NOTE

`axis=-1` **means "the last axis"**, so it works for any number of features, not only 2. For points with 300 features (word vectors) the shapes would be (N, N, 300) and the code would stay the same.

Move 4: glue it into one line and check it against the loops

PYTHON • type this in Colab

```
P = np.array([[0, 0], [3, 4], [1, 1]])

# loop version (Step 5)
loop = np.zeros((3, 3))
for i in range(3):
    for j in range(3):
        loop[i, j] = np.sum((P[i] - P[j]) ** 2)

# one-line broadcast version
fast = np.sum((P[:, np.newaxis, :] - P[np.newaxis, :, :]) ** 2, axis=-1)

print(fast)
print(np.array_equal(loop, fast))      # the same table, no loops
```

▶ OUTPUT

```
[[ 0 25  2]
 [25  0 13]
 [ 2 13  0]]
True
```

MEMORY TRICK

"Column against row, one extra axis for the pair, then add the last axis." Say these three things and you can rebuild the line at any time.

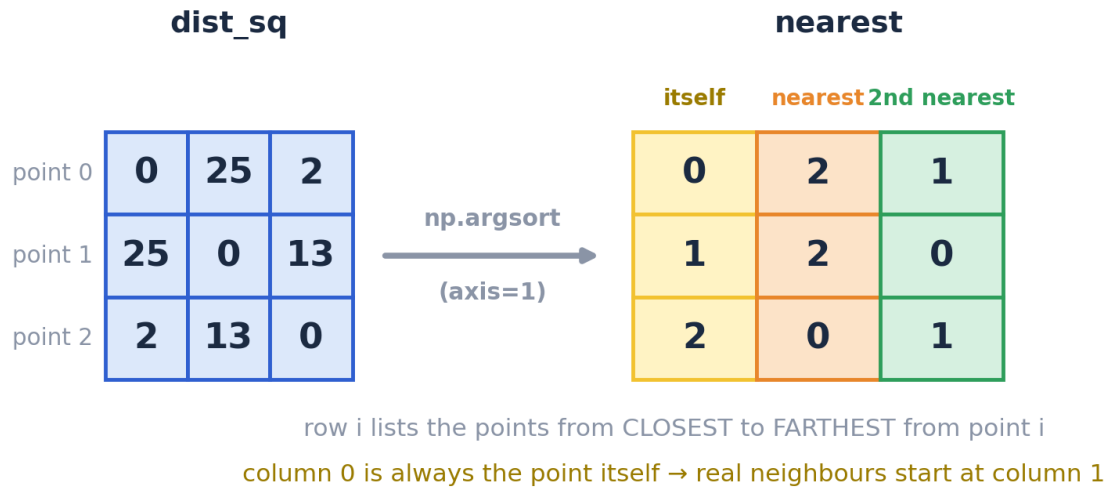
IMPORTANT

If this line still feels heavy, that is normal. It is the combination of broadcasting + `np.newaxis` + `axis=-1` in a single line. The loop version in 3.5 computes exactly the same numbers, so you are never "lost": you can always fall back to the loop to check yourself.

3.7 Step 7: reading the neighbours with argsort

BIG PICTURE

The table says how far every point is from every other point. To find neighbours, **sort every row**. The first entry of each sorted row is the closest point, the second is the second closest, and so on. `np.argsort(..., axis=1)` sorts **each row** and returns the positions, exactly like the race example in Part 2.



Every row of the table is argsorted. The list in row i goes from closest to farthest from point i.

PYTHON • type this in Colab

```
P = np.array([[0, 0], [3, 4], [1, 1]])
dist_sq = np.sum((P[:, np.newaxis, :] - P[np.newaxis, :, :]) ** 2, axis=-1)
nearest = np.argsort(dist_sq, axis=1)
print(nearest)
print(nearest[:, 1])        # column 1 = the single nearest neighbour of every point
```

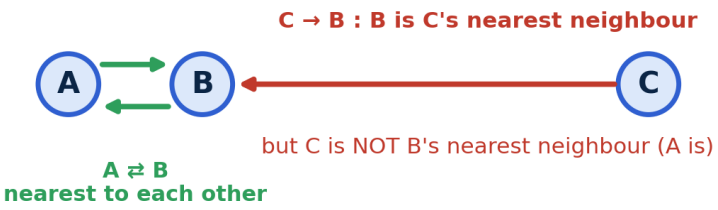
▶ OUTPUT

```
[[0 2 1]
 [1 2 0]
 [2 0 1]]
[2 2 0]
```

Reading row 0: `[0, 2, 1]` means from point 0 the order is itself (0), then point 2, then point 1. Column 0 is **always the point itself**, because its distance to itself is 0, which is the smallest. So the real neighbours start at column 1. For the k nearest neighbours of every point, use `nearest[:, 1:k+1]`.

COMMON TRAP

"A is B's nearest neighbour" does not mean "B is A's nearest neighbour". In the output above, point 1's nearest neighbour is point 2, but point 2's nearest neighbour is point 0. That is why, in the book's plot, some points have more than two lines coming out of them.



"nearest neighbour of" is NOT mutual

C's nearest neighbour is B, but B's nearest neighbour is A. The relationship is not mutual.

3.8 Step 8: skip the full sort with argpartition, and the final function

A full sort puts **every** point in order, but we only want the k closest. From Part 2 you know the cheaper tool: `np.argpartition` pulls the smallest k+1 to the front (the +1 is for the point itself). The order inside those k+1 is not guaranteed, which is fine if you only need to know **who** the neighbours are.

PYTHON • type this in Colab

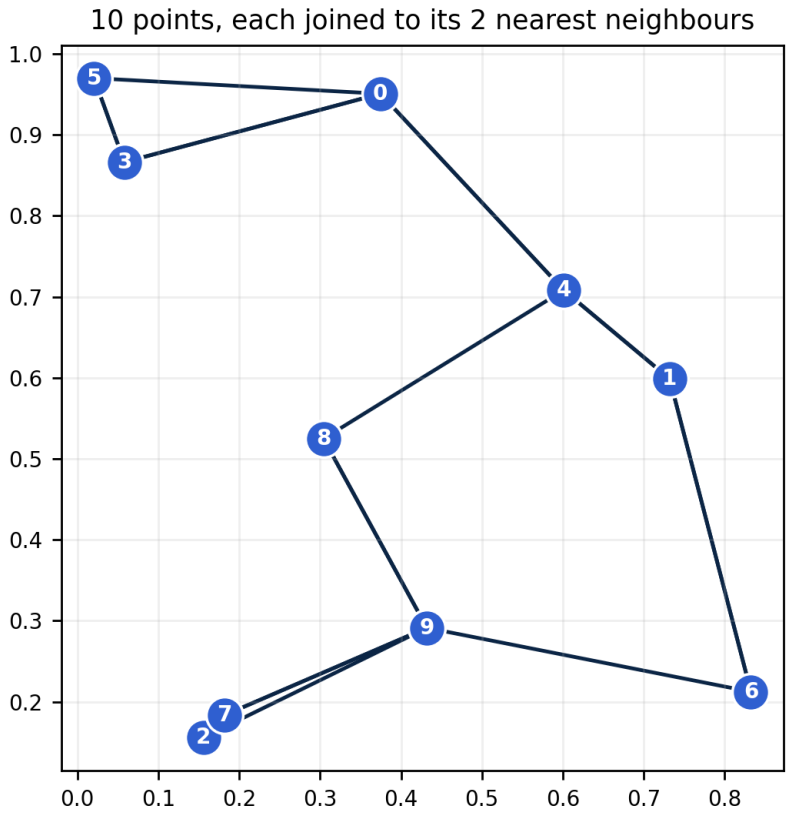
```
rand = np.random.RandomState(42)
X = rand.rand(10, 2)                    # 10 random points in 2D
dist_sq = np.sum((X[:, np.newaxis, :] - X[np.newaxis, :, :]) ** 2, axis=-1)

nearest = np.argsort(dist_sq, axis=1)    # full sort of every row
print(nearest)
print(np.array_equal(nearest[:, 0], np.arange(10)))   # column 0 is always the point itself

K = 2
part = np.argpartition(dist_sq, K + 1, axis=1)[:, :K + 1]   # only the K+1 smallest per row
print(part)
```

▶ OUTPUT

```
[[0 3 4 5 8 1 9 7 2 6]
 [1 4 6 9 8 0 7 3 2 5]
 [2 7 9 8 6 4 3 1 0 5]
 [3 5 0 8 4 9 7 2 1 6]
 [4 1 0 8 9 6 3 5 7 2]
 [5 3 0 8 4 9 1 7 2 6]
 [6 1 9 4 8 7 2 0 3 5]
 [7 2 9 8 6 4 1 3 0 5]
 [8 9 4 7 2 3 0 1 5 6]
 [9 8 7 2 6 1 4 0 3 5]]
True
[[0 3 4]
 [1 4 6]
 [2 7 9]
 [3 5 0]
 [4 1 0]
 [5 3 0]
 [6 1 9]
 [7 2 9]
 [8 9 4]
 [9 8 7]]
```



The 10 points, each joined to its 2 nearest neighbours (the picture the book draws).

Now a small reusable function that does everything from this part. Read it line by line; every line is something you built above:

PYTHON • type this in Colab

```
def k_nearest(X, k):
    # squared distance between every pair of points (N x N table)
    dist_sq = np.sum((X[:, np.newaxis, :] - X[np.newaxis, :, :]) ** 2, axis=-1)
    # sort every row: closest first. Column 0 is the point itself, so skip it
    order = np.argsort(dist_sq, axis=1)
    return order[:, 1:k + 1]

rand = np.random.RandomState(42)
X = rand.rand(10, 2)
print(k_nearest(X, 2))       # the 2 nearest neighbours of each of the 10 points
```

▶ OUTPUT

```
[[3 4]
 [4 6]
 [7 9]
 [5 0]
 [1 0]
 [3 0]
 [1 9]
 [2 9]
 [9 4]
 [8 7]]
```

3.9 Why this matters, and its limits

WHY IT MATTERS

Semantic search and RAG. When a chatbot looks for the document chunks closest to your question, it turns the question and every chunk into vectors (embeddings) and finds the **k closest vectors**. That is Steps 1 to 3 of this part, just with 300 to 1,500 numbers per vector instead of 2. Real systems often use **cosine similarity** instead of squared distance (look it up later), but the shape of the code is the same.

Limit	Why	What real systems do
Memory	the (N, N, D) block must fit in RAM. N = 10,000 points with D = 2 means 10,000 × 10,000 × 2 = 200 million numbers × 8 bytes = **about 1.6 GB**	work in chunks, or use specialised libraries
Time	brute force compares every pair: about **N^2 steps**	tree-based or approximate methods that scale about N log N (e.g. the KD-tree in scikit-learn)
Speed of sorting	a full sort of every row does more work than needed	`np.argpartition`, as in 3.8

INTERVIEW POINT

Say this: "Brute-force KNN builds a pairwise distance matrix using broadcasting, then finds neighbours by sorting or partitioning each row. It is $O(N^2)$ in time and memory, which is why large systems use KD-trees or approximate nearest-neighbour search."

PART 4

4 Big-O: How Work Grows

Aside from Block 8: the one idea you need to compare algorithms

IN THIS PART YOU WILL LEARN

- ✓ What Big-O means in plain words (the **growth**, not the seconds)
- ✓ The four growth rates you will actually meet: **N, N log N, N², N × N!**
- ✓ Why **N²** appears whenever you compare "everything with everything" (like our distance table)
- ✓ The honest limit of Big-O: on **small data** the "worse" algorithm can win

4.1 The big idea

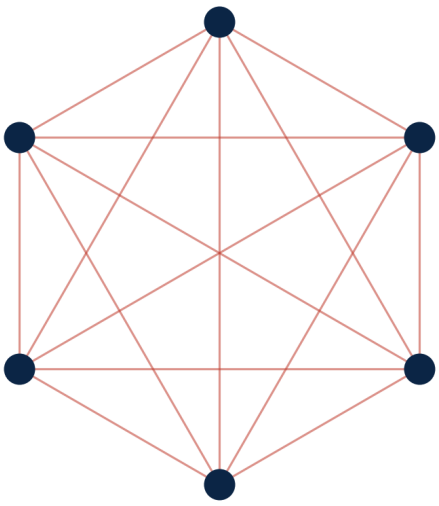
BIG PICTURE

Big-O answers one question: if I give the algorithm more data, how much more work does it do? It does **not** tell you the seconds. It tells you the **shape of the growth**. Two algorithms might both take 1 second today, but one may take 5 seconds when the data grows 5 times, and the other 25 seconds. Big-O predicts that difference before you run anything.

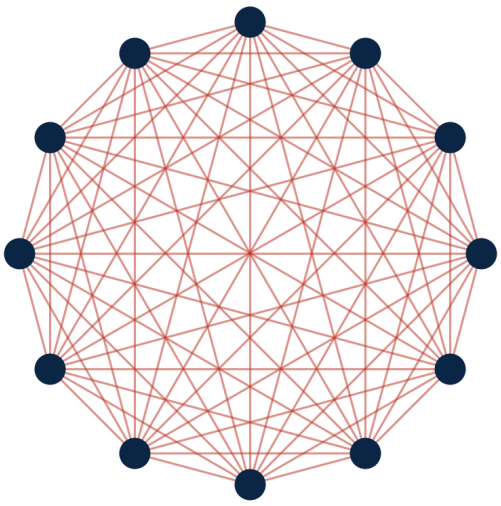
ANALOGY

Three jobs at a party of N people. (1) Hand each person a leaflet: you meet each person once, so the work is about **N**. (2) Make everyone shake hands with everyone: the work grows like **N²**. (3) Find a name in a sorted phone book by repeatedly halving the pages: the work grows very slowly, like **log N**.

N = 6 people → 15 handshakes
(N × N = 36 table entries)



N = 12 people → 66 handshakes
(N × N = 144 table entries)

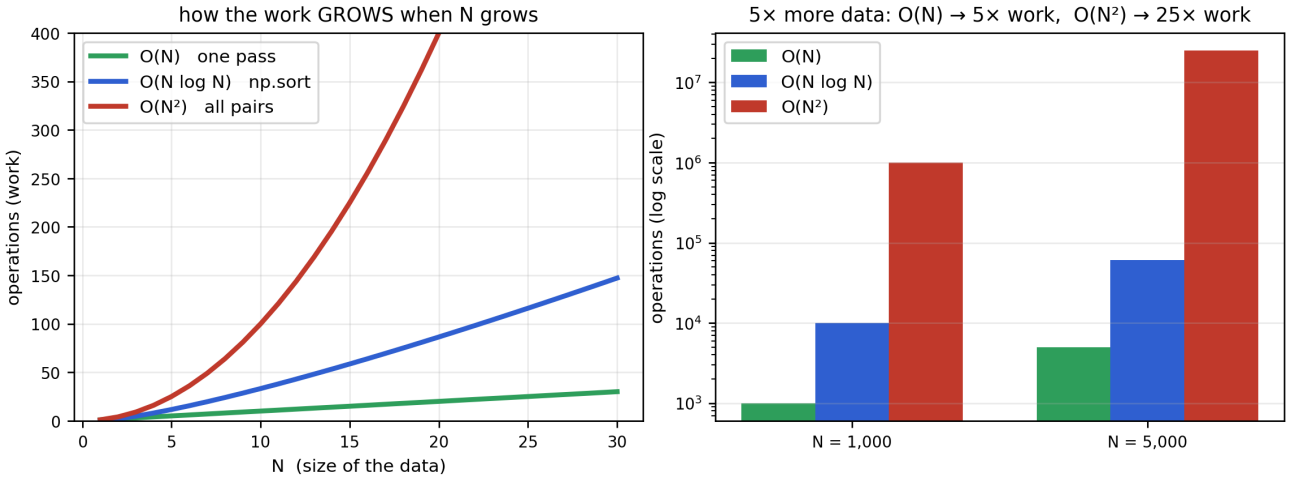


Everyone shakes hands with everyone. Doubling the people from 6 to 12 takes the handshakes from 15 to 66, and the N × N table from 36 to 144.

That is exactly our distance table from Part 3: N points, N × N entries, because every point is compared with every point. Whenever you see "everything against everything", think **N²**.

4.2 The growth rates you will meet

Growth	Say it as	N: 1,000 → 5,000	Where you meet it
N	order N	work × 5	one pass over the data (a sum, a mean)
N log N	order N log N	work × a bit over 5	`np.sort`, `np.argsort`
N^2	order N squared	work × 25	selection sort, brute-force KNN, the distance table
N × N!	order N times N factorial	impossible	bogosort: never use it



Left: the red N^2 curve leaves the others behind. Right: 5× more data costs 5× more work for O(N) but 25× more for $O(N^2)$.

Work out the real numbers for three data sizes. `N * np.log2(N)` is the N log N column; `N ** 2` is the N^2 column:

PYTHON • type this in Colab

```
for N in [10, 1_000, 1_000_000]:
    print(f"N={N:>9,}   N log N={N * np.log2(N):>14,.0f}   N^2={N ** 2:>16,}")
```

▶ OUTPUT

```
N=       10   N log N=            33   N^2=             100
N=    1,000   N log N=         9,966   N^2=       1,000,000
N=1,000,000   N log N=    19,931,569   N^2=1,000,000,000,000
```

At N = 1,000,000, N log N is about 20 million steps while N^2 is **one trillion** steps. That is roughly **50,000 times** more work, which is why choosing the right algorithm matters far more than choosing fast hardware.

MEMORY TRICK

Double the data: O(N) → 2× work. $O(N^2)$ → **4×** work. O(N log N) → a little over 2× work. **Ten times the data:** $O(N^2)$ → **100×** work.

4.3 The honest limit of Big-O

IMPORTANT

Big-O tells you how work **scales**, not how long a **single run** takes. For small data, an algorithm with worse scaling can be faster. Example: an $O(N^2)$ algorithm that takes 0.01 seconds can beat an O(N) algorithm that takes 1 second, because the O(N) one has a heavy fixed cost. Scale N up by 1,000 and the O(N) algorithm wins easily. This is the same lesson as the book's histogram timing: judge an algorithm by both its scaling and your real data size.

WHY IT MATTERS	**Self-attention in transformers compares every token with every other token**, so its work grows like N^2 with the sequence length. That is why long context windows are expensive. It is the same "everything against everything" table you built in Part 3, just with tokens instead of points. You will meet this again in the LLM stages of your roadmap.
INTERVIEW POINT	Say this: "Big-O describes how an algorithm's running time or memory grows with input size, ignoring constants. O(N log N) sorting scales far better than $O(N^2)$ pairwise comparison, but on very small inputs the constant factors can reverse the ranking."
NOTE	The book also mentions big-Θ, big-Ω and small-o notation. In data-science practice people use "Big-O" loosely to mean "how it scales", and that loose meaning is all you need for ML work. The formal versions matter for theory exams, not for your NumPy code.

5

PART 5

Revision Zone

Cheat sheets for the day before an exam or interview

IN THIS PART YOU WILL LEARN

- ✓ A function cheat sheet for Blocks 7 and 8
- ✓ The trap radar: check these before trusting your output
- ✓ A topic checklist and the list of what was left out on purpose

5.1 Function cheat sheet

Function / pattern	What it does	Priority
`x[[3, 7, 4]]`	pick positions from a list; result follows the index shape	MUST KNOW
`X[row, col]`	pairs of (row, col), like zip; 1D result	MUST KNOW
`X[row[:, None], col]`	grid: every row with every column	MUST KNOW
`X[1:, [2, 0, 1]]`	slice one axis, fancy-index the other	MUST KNOW
`X[:, mask]`	keep the columns where the Boolean mask is True	MUST KNOW
`probs[np.arange(n), labels]`	probability of the true class for every sample	MUST KNOW
`np.random.choice(n, k, replace=False)`	k different random numbers from 0..n-1	MUST KNOW
`np.random.permutation(n)`	shuffled 0..n-1, each once (for splits)	MUST KNOW
`np.add.at(x, i, 1)`	add for EVERY occurrence of an index (counting)	GOOD TO KNOW
`np.bincount(ids)`	count how often each integer appears	GOOD TO KNOW
`np.searchsorted(bins, vals)`	bucket index for every value	GOOD TO KNOW
`np.sort(x) / x.sort()`	sorted copy / sort in place (returns None)	MUST KNOW
`np.argsort(x)`	positions that would sort x; use as a fancy index	MUST KNOW
`np.argsort(-probs)[:k]`	positions of the k biggest values	MUST KNOW
`np.sort(X, axis=1)`	sort inside each row (axis=0: each column)	MUST KNOW
`np.partition(x, k)`	k smallest on the left, order inside not guaranteed	GOOD TO KNOW
`np.argpartition(x, -k)[-k:]`	positions of the k biggest, without a full sort	GOOD TO KNOW
`np.sum((A[:, None, :] - A[None, :, :])**2, axis=-1)`	N × N table of squared distances	GOOD TO KNOW
`np.ix_(rows, cols)`	builds the row/column grid index for you	LOOK UP

5.2 The trap radar

Trap	What goes wrong	Fix
Slice vs fancy	fancy indexing returns a **copy**; editing it does not change the original	know which one you made; slices are views
Pairs vs grid	`X[row, col]` gives a list, not a table	make the row index a column: `row[:, None]`
Integer vs Boolean index	the dtype decides: integers are positions, Booleans are yes/no votes	print the index array and its `dtype` first
`x[i] += 1` with repeats	repeated positions collapse; counts come out wrong	`np.add.at(x, i, 1)` or `np.bincount`
`replace=False` and splits	two separate random draws can overlap (data leakage)	one `permutation`, cut it in two
`x = x.sort()`	in-place methods return `None`, so you lose your data	use `np.sort(x)` or call `x.sort()` alone
`np.sort(X, axis=...)` on rows	each row/column is sorted alone, so rows stop belonging together	`order = np.argsort(X[:, 0]); X[order]`
Partition inside order	inside the left and right groups the order is not promised	sort only the k winners if you need order
Neighbour is not mutual	A is B's nearest neighbour does not mean the reverse	do not assume symmetry
KNN column 0	column 0 of the argsorted table is the point itself	take `nearest[:, 1:k+1]`

5.3 Topic checklist

Tick these mentally. If you can explain one in two sentences, in plain words, you own it.

- ☐ Fancy indexing = a list of positions; the result takes the shape of the index array.
- ☐ Slices are views; fancy indexing returns copies.
- ☐ `X[row, col]` makes pairs; a column-vector row index makes a grid.
- ☐ Integer arrays are positions; Boolean arrays are yes/no votes (the dtype decides).
- ☐ A random train/test split uses one shuffled list so no row is in both sets.
- ☐ `x[i] += 1` collapses repeats; `np.add.at` accumulates them.
- ☐ `searchsorted` gives the bucket index, `add.at` counts per bucket.
- ☐ `sort` returns values; `argsort` returns positions; `axis` picks the direction of the sort.
- ☐ `partition` is a cheap "top-k"; the order inside the two sides is not promised.
- ☐ KNN = distance to all points, sort, take the first k, vote.
- ☐ The distance table is a column of points against a row of points, with an extra axis for the pair, then `sum(axis=-1)`.
- ☐ Big-O describes how work grows with data; "everything against everything" is N^2.

5.4 Left out on purpose, so nothing is hidden

Left out	Why
`selection_sort` and `bogosort` code	DSA practice, not a NumPy skill. We kept only the scaling idea (Part 4).
`np.histogram` vs custom speed benchmark	results depend on your machine; the lesson is kept in a note (1.9).
`plt.hist`, `plt.scatter` lines	Matplotlib is a later topic; we use only the pictures.
`reduceat` (mentioned by the book)	rarely used; look it up if you ever need it.
Big-Θ, Big-Ω, small-o	theory-exam detail; the loose "Big-O" meaning is enough for ML.

NOTE

Next: Block 9 is **Structured Data: NumPy's Structured Arrays**. We will first check how much of it you really need, because in practice most structured data work is done in pandas.